

BMI500 Homework 3: Code Profiling

Jasmine Zhou

Department of Computer Science, Emory University

September 14, 2026

Coarse-grained Profiling

PBMC3k Console Output (selected)

- Time: 23.96s for user, 1s for system
- CPU: 96%
- Memory: 865,280 kB

```
User time (seconds): 23.96
System time (seconds): 1.00
Percent of CPU this job got: 96%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:25.94
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 865280
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 2
Minor (reclaiming a frame) page faults: 163432
Voluntary context switches: 9259
Involuntary context switches: 99
Swaps: 0
File system inputs: 19848
File system outputs: 87760
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0
```

PBMC6k Console Output (selected)

- Time: 34.02s for user, 1.35s for system
- CPU: 93%
- Memory: 887,172 kB

```
User time (seconds): 34.02
```

System time (seconds): 1.35
Percent of CPU this job got: 93%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:37.68
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 887172
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 2444
Minor (reclaiming a frame) page faults: 203694
Voluntary context switches: 16520
Involuntary context switches: 111
Swaps: 0
File system inputs: 581856
File system outputs: 160432
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0

PBMC10k Console Output (selected)

- Time: 75.07s for user, 7.96s for system
- CPU: 97%
- Memory: 1,878,476 kB

User time (seconds): 75.07
System time (seconds): 7.96
Percent of CPU this job got: 97%
Elapsed (wall clock) time (h:mm:ss or m:ss): 1:24.91
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 1878476
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 2527
Minor (reclaiming a frame) page faults: 845885
Voluntary context switches: 27256
Involuntary context switches: 310
Swaps: 0
File system inputs: 1197936
File system outputs: 831672
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0

Instrumental Profiling

Major bottleneck across different datasets: startup_argparse, neighbors, umap, and rank_genes_groups.

- startup_argparse: measures startup/import overhead
- neighbors: builds a k-nearest neighbor graph in the 30-dimensional PCA space,
- umap: runs UMAP on that neighbor graph
- rank_genes_groups: a Wilcoxon rank-sum test of each gene in each Louvain cluster against all other cells, to find marker genes

PBMC3k Console Output

```
=== Timing summary per section ===
```

Function	Time	Percentage of total time
startup_argparse	7.179 s	(31.8%)
read_10x_mtx	0.154 s	(0.7%)
filter_cells_genes	0.056 s	(0.2%)
normalize_log1p	0.044 s	(0.2%)
highly_variable_genes	0.058 s	(0.3%)
scale	0.043 s	(0.2%)
pca	0.312 s	(1.4%)
neighbors	4.244 s	(18.8%)
louvain	0.134 s	(0.6%)
umap	8.332 s	(36.9%)
write_h5ad	0.149 s	(0.7%)
rank_genes_groups	1.857 s	(8.2%)
TOTAL	22.562 s	

PBMC6k Console Output

```
=== Timing summary per section ===
```

Function	Time	Percentage of total time
startup_argparse	7.362 s	(22.1%)
read_10x_mtx	0.211 s	(0.6%)
filter_cells_genes	0.088 s	(0.3%)
normalize_log1p	0.073 s	(0.2%)
highly_variable_genes	0.089 s	(0.3%)
scale	0.080 s	(0.2%)
pca	0.592 s	(1.8%)
neighbors	4.380 s	(13.2%)
louvain	0.374 s	(1.1%)
umap	16.114 s	(48.4%)
write_h5ad	0.268 s	(0.8%)
rank_genes_groups	3.655 s	(11.0%)
TOTAL	33.285 s	

PBMC10k Console Output

```
=== Timing summary per section ===
```

Function	Time	Percentage of total time
startup_argparse	7.298 s	(9.1%)
read_10x_mtx	8.008 s	(10.0%)
filter_cells_genes	0.427 s	(0.5%)
normalize_log1p	0.452 s	(0.6%)
highly_variable_genes	0.466 s	(0.6%)
scale	0.176 s	(0.2%)
pca	1.092 s	(1.4%)
neighbors	31.748 s	(39.6%)
louvain	0.795 s	(1.0%)
umap	14.165 s	(17.7%)
write_h5ad	0.701 s	(0.9%)
rank_genes_groups	14.759 s	(18.4%)
TOTAL	80.087 s	

Per-section Runtime by Dataset

As the number of cells increases, neighbors becomes the main bottleneck and jumps to around 40% of the time. Gene ranking also increased substantially, while UMAP increased but not consistently. Read matrix becomes more significant at large dataset sizes. Startup and argparse remain mostly fixed across datasets. The other functions remain insignificant/not noticeable.

At 20,000 cells, some sections' runtime should remain similar, but others will increase dramatically. Startup_argparse should remain similar because the overhead is mainly fixed. Read_10_mtx runtime will increase slightly because we are processing a larger matrix. PCA will increase slightly due to higher dimensions. Neighbors and rank_genes_groups serve as the main bottleneck and should double pbmc10k's runtime (takes around 68s and 30s, respectively). UMAP should slightly increase in runtime or be similar to 10k. Louvain and Other should increase slightly. Given the total runtime of pbmc10k is around 80 seconds, a 20,000-cell dataset should take around 145 to 150 seconds to run.

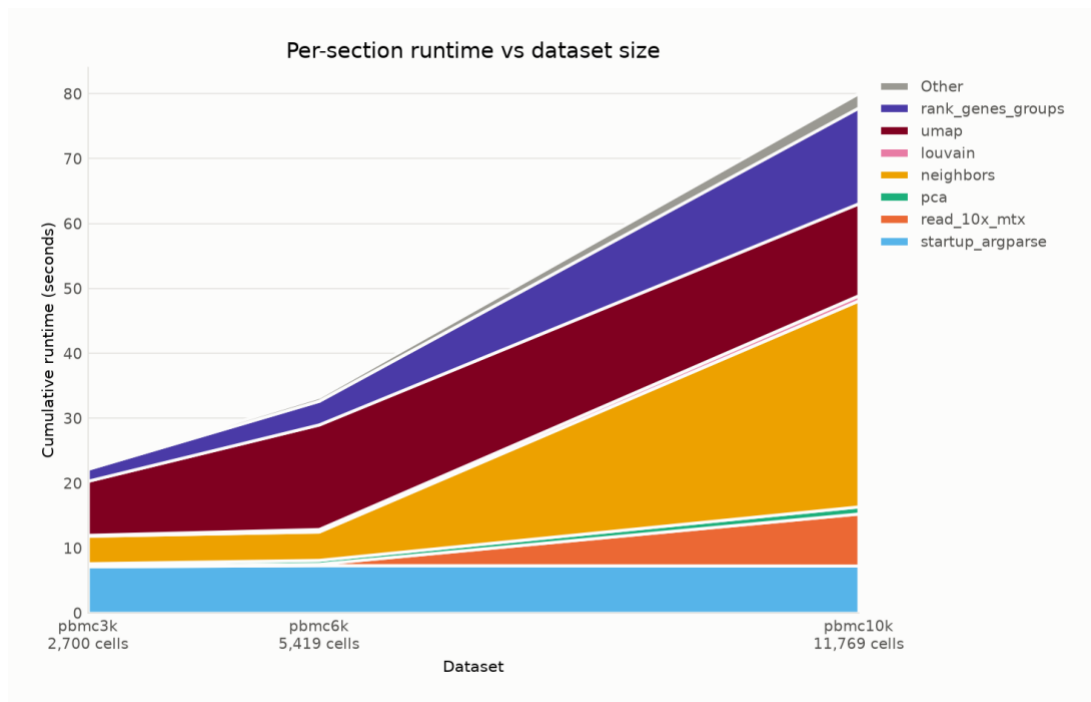


Figure 1 Runtime per Section vs. PBMC datasets (pbmc3k, pbmc6k, and pbmc10k). Each section's runtime is represented by its band thickness.

Fine-grained Profiling (cProfile)

Console output

The original cProfile console outputs contained over 1000 lists for each dataset and would be inappropriate for display. The reduced version (top 50 lists) is available in the Supplemental Materials section.

cProfile Visualizations

The following cProfile visualizations are generated using SnakeViz from the saved .prof files in cProfiling. Separate cProfile runs disabled tqdm's background monitor to prevent inconsistent caller records and produce SnakeViz visualizations. Coarse and instrumented timings were retained from the original experiments.

PBMC3k cProfile

SnakeViz

Reset Root

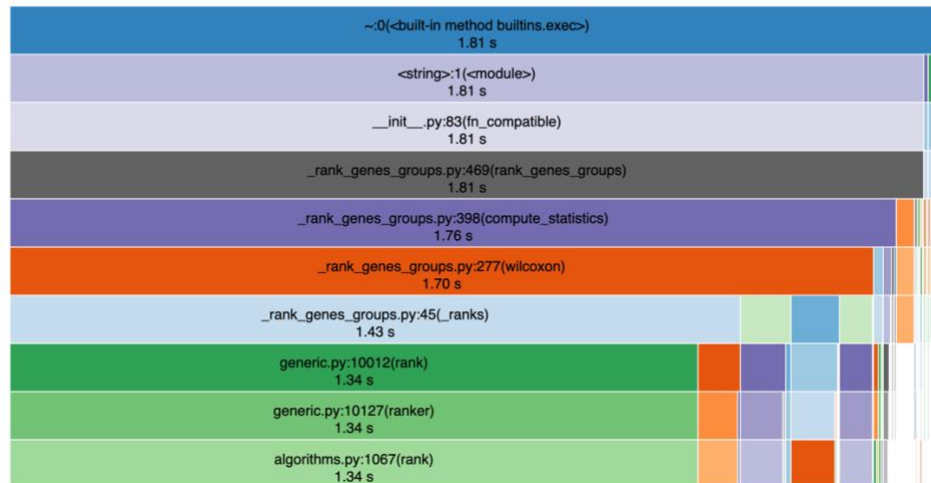
Reset Zoom

Style: **Icicle** ▾

Depth: **10** ▾

Cutoff: **1 / 1000** ▾

Call Stack



PBMC6k cProfile

SnakeViz

Reset Root

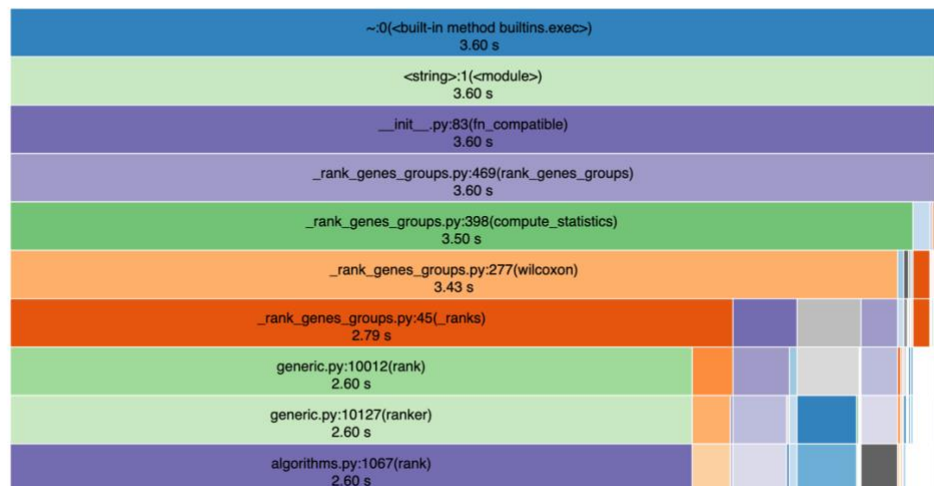
Reset Zoom

Style: **Icicle** ▾

Depth: **10** ▾

Cutoff: **1 / 1000** ▾

Call Stack



PBMC10k cProfile

SnakeViz

Reset Root

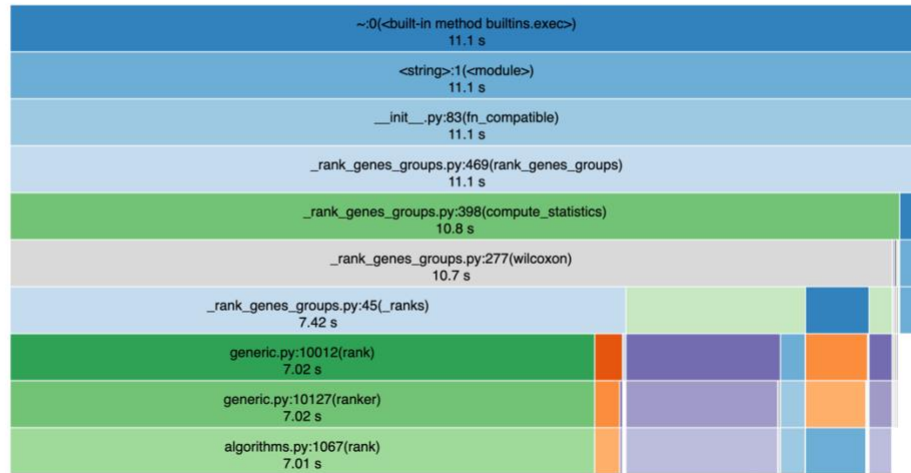
Reset Zoom

Style: **Icicle** ▾

Depth: **10** ▾

Cutoff: **1 / 1000** ▾

Call Stack



SLURM Scripts

Below are the SLURM scripts for running each experiment with the three datasets. Each experiment contains coarse-grained profiling, instrumental profiling, and fine-grained profiling with cProfile.

PBMC3k Slurm Script

```
#!/bin/bash
#SBATCH --job-name=prof-pbmc3k
#SBATCH --partition=batch
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --mem=16G
#SBATCH --time=00:30:00
#SBATCH --output=logs/profile_pbmc3k_%j.out
#SBATCH --error=logs/profile_pbmc3k_%j.err
```

```
# Submit: sbatch profile_pbmc3k.sbatch
```

```
set -euo pipefail
```

```
DS=pbmc3k
```

```
HW=/home/rzhou73/BMI500/HW3
```

```

cd "$HW"

mkdir -p logs profiles

source /home/rzhou73/bmi500/bin/activate

# Pin every threading layer to 1 so timings are comparable across datasets
export OMP_NUM_THREADS=1
export OPENBLAS_NUM_THREADS=1
export MKL_NUM_THREADS=1
export NUMBA_NUM_THREADS=1

COARSE="profiles/${DS}_coarse_time.txt"
INSTR="profiles/${DS}_instrumented_time.txt"
FINE="profiles/${DS}_rank_genes_groups_profile.txt"

echo "=====
echo " dataset : $DS"
echo " node   : $(hostname)"
echo " job    : ${SLURM_JOB_ID}"
echo " cpus   : ${SLURM_CPUS_PER_TASK}   mem: 16G"
echo " started : $(date)"
echo "=====

# Extract data if not decompressed yet
if [[ ! -d "data/${DS}/filtered_gene_bc_matrices" ]]; then
    echo "[setup] extracting data/${DS}.tgz"
    tar xzf "data/${DS}.tgz" -C data/
fi

# THE MEASURED RUN
echo
echo "##### RUN: scanpy_pbmc.py under /usr/bin/time -v #####"
/usr/bin/time -v -o "$COARSE" python scanpy_pbmc.py --data-set "$DS"

# ALL THREE EXPERIMENTS
echo
echo "##### 1. COARSE-GRAINED ($COARSE) #####"

```

```
cat "$COARSE"
```

```
echo
```

```
echo "##### 2. INSTRUMENTED ($INSTR) #####"
```

```
cat "$INSTR"
```

```
echo
```

```
echo "##### 3. FINE-GRAINED ($FINE) #####"
```

```
head -20 "$FINE"
```

```
echo
```

```
echo "finished : $(date)"
```

```
ls -lh profiles/${DS}_* 2>/dev/null || true
```

PBMC6k Slurm Script

```
#!/bin/bash
```

```
#SBATCH --job-name=prof-pbmc6k
```

```
#SBATCH --partition=batch
```

```
#SBATCH --nodes=1
```

```
#SBATCH --ntasks=1
```

```
#SBATCH --cpus-per-task=4
```

```
#SBATCH --mem=24G
```

```
#SBATCH --time=00:45:00
```

```
#SBATCH --output=logs/profile_pbmc6k_%j.out
```

```
#SBATCH --error=logs/profile_pbmc6k_%j.err
```

```
# Profiling run for pbmc6k (5,419 cells)
```

```
set -euo pipefail
```

```
DS=pbmc6k
```

```
HW=/home/rzhou73/BMI500/HW3
```

```
cd "$HW"
```

```
mkdir -p logs profiles
```

```
source /home/rzhou73/bmi500/bin/activate
```

```

# Pin every threading layer to 1 so timings are comparable across datasets
export OMP_NUM_THREADS=1
export OPENBLAS_NUM_THREADS=1
export MKL_NUM_THREADS=1
export NUMBA_NUM_THREADS=1

COARSE="profiles/${DS}_coarse_time.txt"
INSTR="profiles/${DS}_instrumented_time.txt"
FINE="profiles/${DS}_rank_genes_groups_profile.txt"

echo "=====
echo " dataset : $DS"
echo " node   : $(hostname)"
echo " job    : ${SLURM_JOB_ID}"
echo " cpus   : ${SLURM_CPUS_PER_TASK}  mem: 24G"
echo " started : $(date)"
echo "=====

if [[ ! -d "data/${DS}/filtered_gene_bc_matrices" ]]; then
    echo "[setup] extracting data/${DS}.tgz"
    tar xzf "data/${DS}.tgz" -C data/
fi

# THE MEASURED RUN
echo
echo "##### RUN: scanpy_pbmc.py under /usr/bin/time -v #####"
/usr/bin/time -v -o "$COARSE" python scanpy_pbmc.py --data-set "$DS"

# ALL THREE EXPERIMENTS
echo
echo "##### 1. COARSE-GRAINED ($COARSE) #####"
cat "$COARSE"

echo
echo "##### 2. INSTRUMENTED ($INSTR) #####"

```

```
cat "$INSTR"
```

```
echo
```

```
echo "##### 3. FINE-GRAINED ($FINE) #####"
```

```
head -20 "$FINE"
```

```
echo
```

```
echo "finished : $(date)"
```

```
ls -lh profiles/${DS}_* 2>/dev/null || true
```

PBMC10k Slurm Script

```
#!/bin/bash
```

```
#SBATCH --job-name=prof-pbmc10k
```

```
#SBATCH --partition=batch
```

```
#SBATCH --nodes=1
```

```
#SBATCH --ntasks=1
```

```
#SBATCH --cpus-per-task=4
```

```
#SBATCH --mem=64G
```

```
#SBATCH --time=02:00:00
```

```
#SBATCH --output=logs/profile_pbmc10k_%j.out
```

```
#SBATCH --error=logs/profile_pbmc10k_%j.err
```

```
# Profiling run for pbmc10k
```

```
set -euo pipefail
```

```
DS=pbmc10k
```

```
HW=/home/rzhou73/BMI500/HW3
```

```
cd "$HW"
```

```
mkdir -p logs profiles
```

```
source /home/rzhou73/bmi500/bin/activate
```

```
# Pin every threading layer to 1 so timings are comparable across datasets
```

```
export OMP_NUM_THREADS=1
```

```
export OPENBLAS_NUM_THREADS=1
```

```
export MKL_NUM_THREADS=1
```

```
export NUMBA_NUM_THREADS=1
```

```
COARSE="profiles/${DS}_coarse_time.txt"
```

```
INSTR="profiles/${DS}_instrumented_time.txt"
```

```
FINE="profiles/${DS}_rank_genes_groups_profile.txt"
```

```
echo "=====
```

```
echo " dataset : $DS (11,769 cells)"
```

```
echo " node   : $(hostname)"
```

```
echo " job    : ${SLURM_JOB_ID}"
```

```
echo " cpus   : ${SLURM_CPUS_PER_TASK} mem: 64G"
```

```
echo " started : $(date)"
```

```
echo "=====
```

```
if [[ ! -d "data/${DS}/filtered_gene_bc_matrices" ]]; then
```

```
    echo "[setup] extracting data/${DS}.tgz"
```

```
    tar xzf "data/${DS}.tgz" -C data/
```

```
fi
```

```
# THE MEASURED RUN
```

```
echo
```

```
echo "##### RUN: scanpy_pbmc.py under /usr/bin/time -v #####"
```

```
/usr/bin/time -v -o "$COARSE" python scanpy_pbmc.py --data-set "$DS"
```

```
# ALL THREE EXPERIMENTS
```

```
echo
```

```
echo "##### 1. COARSE-GRAINED ($COARSE) #####"
```

```
cat "$COARSE"
```

```
echo
```

```
echo "##### 2. INSTRUMENTED ($INSTR) #####"
```

```
cat "$INSTR"
```

```
echo
```

```
echo "##### 3. FINE-GRAINED ($FINE) #####"
```

```
head -20 "$FINE"
```

```
echo
```

```
echo "finished : $(date)"
```

```
ls -lh profiles/${DS}_* 2>/dev/null || true
```

Supplemental Materials

PBMC3k Console Output (top 50 lines)

Tue Sep 15 15:35:33 2026 profiles/pbmc3k_rank_genes_groups.prof

42155 function calls (40996 primitive calls) in 1.811 seconds

Ordered by: internal time

List reduced from 1031 to 50 due to restriction <50>

ncalls	totttime	percall	cumtime	percall	filename:lineno(function)
4	1.337	0.334	1.337	0.334	algorithms.py:1067(rank)
53	0.089	0.002	0.089	0.002	take.py:120(_take_nd_ndarray)
4	0.077	0.019	0.077	0.019	{built-in method
scipy.sparse._sparsetools.csr_todense}					
8	0.076	0.010	0.076	0.010	{built-in method
scipy.sparse._sparsetools.csr_column_index2}					
1	0.035	0.035	0.035	0.035	
__init__.py:781(_check_nonnegative_integers_in_mem)					
270	0.034	0.000	0.034	0.000	{method 'reduce' of
'numpy.ufunc' objects}					
22	0.018	0.001	0.018	0.001	missing.py:261(_isna_array)
5	0.014	0.003	1.435	0.287	
_rank_genes_groups.py:45(_ranks)					
8	0.010	0.001	0.010	0.001	{built-in method
scipy.sparse._sparsetools.csr_column_index1}					
8	0.010	0.001	0.010	0.001	
_utils.py:47(sparse_mean_variance_axis)					
23	0.009	0.000	0.009	0.000	{method 'argsort' of
'numpy.ndarray' objects}					
5	0.006	0.001	1.704	0.341	
_rank_genes_groups.py:277(wilcoxon)					
90	0.005	0.000	0.006	0.000	
numeric.py:1330(normalize_axis_tuple)					
16	0.005	0.000	0.065	0.004	frame.py:11462(_reduce)
4	0.005	0.001	0.005	0.001	{built-in method
scipy.sparse._sparsetools.get_csr_submatrix}					
23	0.004	0.000	0.004	0.000	{built-in method _io.open_code}
99	0.004	0.000	0.004	0.000	{built-in method posix.stat}
328	0.003	0.000	0.003	0.000	{built-in method __new__ of
type object at 0x1500f624dfe0}					
1	0.003	0.003	1.755	1.755	
_rank_genes_groups.py:398(compute_statistics)					
23	0.002	0.000	0.002	0.000	{method '__exit__' of
'_io._IOBase' objects}					
1	0.002	0.002	0.002	0.002	{built-in method
scipy.sparse._sparsetools.csr_eliminate_zeros}					
6276/6249	0.002	0.000	0.004	0.000	{built-in method
builtins.isinstance}					
5	0.002	0.000	0.002	0.000	{built-in method posix.listdir}
92	0.001	0.000	0.001	0.000	base.py:5385(__getitem__)
23	0.001	0.000	0.001	0.000	{built-in method marshal.loads}
20	0.001	0.000	0.002	0.000	multi.py:1194(_engine)
23	0.001	0.000	0.001	0.000	{method 'read' of
'_io.BufferedReader' objects}					

3093/2508	0.001	0.000	0.001	0.000	{built-in method builtins.len}
16	0.001	0.000	0.017	0.001	nanops.py:1499(_maybe_null_out)
45	0.001	0.000	0.007	0.000	function_base.py:5369(insert)
4	0.001	0.000	0.001	0.000	
_continuous_distns.py:370(_norm_cdf)					
52	0.001	0.000	0.002	0.000	multi.py:3001(get_loc)
60	0.001	0.000	0.003	0.000	
construction.py:517(sanitize_array)					
84	0.001	0.000	0.001	0.000	generic.py:6258(__finalize__)
56	0.001	0.000	0.001	0.000	{method 'copy' of
'numpy.ndarray' objects}					
27	0.001	0.000	0.003	0.000	base.py:475(__new__)
79	0.001	0.000	0.001	0.000	managers.py:180(blknos)
40	0.001	0.000	0.001	0.000	{method 'take' of
'numpy.ndarray' objects}					
170	0.001	0.000	0.001	0.000	base.py:650(_simple_new)
20	0.001	0.000	0.001	0.000	
_sputils.py:264(get_index_dtype)					
4	0.000	0.000	0.002	0.000	
_distn_infrastructure.py:2216(sf)					
1049	0.000	0.000	0.001	0.000	generic.py:42(_instancecheck)
94	0.000	0.000	0.001	0.000	generic.py:281(__init__)
200	0.000	0.000	0.000	0.000	{built-in method numpy.empty}
1660	0.000	0.000	0.000	0.000	{built-in method
builtins.getattr}					
5	0.000	0.000	0.003	0.001	records.py:588(fromarrays)
128/5	0.000	0.000	0.001	0.000	{built-in method
_abc._abc_subclasscheck}					
1049	0.000	0.000	0.001	0.000	generic.py:37(_check)
23	0.000	0.000	0.001	0.000	
utils.py:239(maybe_convert_indices)					
4	0.000	0.000	0.005	0.001	multitest.py:99(multipletests)

PBMC6k Console Output (top 50 lines)

Tue Sep 15 15:44:58 2026 profiles/pbmc6k_rank_genes_groups.prof

80020 function calls (78229 primitive calls) in 2.674 seconds

Ordered by: internal time

List reduced from 1031 to 50 due to restriction <50>

ncalls	totttime	percall	cumtime	percall	filename:lineno(function)
8	2.024	0.253	2.024	0.253	algorithms.py:1067(rank)
108	0.140	0.001	0.141	0.001	take.py:120(_take_nd_ndarray)
14	0.125	0.009	0.125	0.009	{built-in method
scipy.sparse._sparsetools.csr_column_index2}					
8	0.100	0.012	0.100	0.012	{built-in method
scipy.sparse._sparsetools.csr_todense}					
1	0.045	0.045	0.046	0.046	
__init__.py:781(_check_nonnegative_integers_in_mem)					
693	0.042	0.000	0.042	0.000	{method 'reduce' of
'numpy.ufunc' objects}					
14	0.022	0.002	0.022	0.002	
_utils.py:47(sparse_mean_variance_axis)					

9	0.020	0.002	2.154	0.239	
_rank_genes_groups.py:45(_ranks)					
14	0.020	0.001	0.020	0.001	{built-in method
scipy.sparse._sparsetools.csr_column_index1}					
62	0.019	0.000	0.019	0.000	missing.py:261(_isna_array)
35	0.012	0.000	0.012	0.000	{method 'argsort' of
'numpy.ndarray' objects}					
316	0.006	0.000	0.006	0.000	base.py:650(_simple_new)
8	0.005	0.001	0.005	0.001	{built-in method
scipy.sparse._sparsetools.get_csr_submatrix}					
56	0.005	0.000	0.079	0.001	frame.py:11462(_reduce)
8	0.003	0.000	2.566	0.321	
_rank_genes_groups.py:277(wilcoxon)					
99	0.003	0.000	0.003	0.000	{built-in method posix.stat}
23	0.003	0.000	0.003	0.000	{built-in method _io.open_code}
12783/12753	0.002	0.000	0.004	0.000	{built-in method
builtins.isinstance}					
1	0.002	0.002	0.002	0.002	{built-in method
scipy.sparse._sparsetools.csr_eliminate_zeros}					
632	0.002	0.000	0.002	0.000	{built-in method __new__ of
type object at 0x147436976fe0}					
23	0.001	0.000	0.001	0.000	{method '__exit__' of
'_io._IOBase' objects}					
186	0.001	0.000	0.002	0.000	base.py:5385(__getitem__)
1	0.001	0.001	2.612	2.612	
_rank_genes_groups.py:398(compute_statistics)					
35	0.001	0.000	0.002	0.000	multi.py:1194(_engine)
35	0.001	0.000	0.001	0.000	{built-in method numpy.zeros}
23	0.001	0.000	0.001	0.000	{built-in method marshal.loads}
5783/4587	0.001	0.000	0.001	0.000	{built-in method builtins.len}
56	0.001	0.000	0.023	0.000	nanops.py:1499(_maybe_null_out)
70	0.001	0.000	0.005	0.000	base.py:475(__new__)
7	0.001	0.000	0.001	0.000	
_continuous_distns.py:370(_norm_cdf)					
223	0.001	0.000	0.001	0.000	generic.py:6258(__finalize__)
118	0.001	0.000	0.004	0.000	
construction.py:517(sanitize_array)					
63	0.001	0.000	0.001	0.000	
utils.py:239(maybe_convert_indices)					
120	0.001	0.000	0.001	0.000	{method 'nonzero' of
'numpy.ndarray' objects}					
151	0.001	0.000	0.001	0.000	{method 'copy' of
'numpy.ndarray' objects}					
237	0.001	0.000	0.001	0.000	generic.py:281(__init__)
78	0.001	0.000	0.001	0.000	function_base.py:5369(insert)
2332	0.001	0.000	0.001	0.000	generic.py:42(_instancecheck)
155/154	0.001	0.000	0.162	0.001	indexing.py:1177(__getitem__)
82	0.001	0.000	0.002	0.000	multi.py:3001(get_loc)
56	0.001	0.000	0.001	0.000	
cast.py:123(maybe_convert_platform)					
433	0.001	0.000	0.001	0.000	{built-in method numpy.empty}
83	0.001	0.000	0.001	0.000	{method 'take' of
'numpy.ndarray' objects}					
2332	0.001	0.000	0.001	0.000	generic.py:37(_check)
56	0.001	0.000	0.001	0.000	range.py:1148(take)
7	0.001	0.000	0.002	0.000	
_distn_infrastructure.py:2216(sf)					

64	0.001	0.000	0.144	0.002	
managers.py:642(reindex_indexer)					
7	0.001	0.000	0.007	0.001	multitest.py:99(multipletests)
3370	0.001	0.000	0.001	0.000	{built-in method
builtins.getattr}					
83	0.001	0.000	0.001	0.000	
cast.py:1166(maybe_infer_to_datetimelike)					

PBMC10k Console Output (top 50 lines)

Tue Sep 15 15:40:19 2026 profiles/pbmc10k_rank_genes_groups.prof

323722 function calls (317915 primitive calls) in 11.051 seconds

Ordered by: internal time

List reduced from 1031 to 50 due to restriction <50>

ncalls	totttime	percall	cumtime	percall	filename:lineno(function)
24	7.013	0.292	7.015	0.292	algorithms.py:1067(rank)
32	1.566	0.049	1.566	0.049	{built-in method
scipy.sparse._sparsetools.csr_column_index2}					
481	0.695	0.001	0.700	0.001	take.py:120(_take_nd_ndarray)
24	0.298	0.012	0.298	0.012	{built-in method
scipy.sparse._sparsetools.csr_todense}					
32	0.294	0.009	0.294	0.009	
_utils.py:47(sparse_mean_variance_axis)					
32	0.277	0.009	0.277	0.009	{built-in method
scipy.sparse._sparsetools.csr_column_index1}					
1	0.250	0.250	0.251	0.251	
__init__.py:781(check_nonnegative_integers_in_mem)					
3834	0.129	0.000	0.129	0.000	{method 'reduce' of
'numpy.ufunc' objects}					
25	0.060	0.002	7.416	0.297	
_rank_genes_groups.py:45(_ranks)					
390	0.053	0.000	0.053	0.000	missing.py:261(_isna_array)
71	0.046	0.001	0.046	0.001	{method 'argsort' of
'numpy.ndarray' objects}					
24	0.031	0.001	0.031	0.001	{built-in method
scipy.sparse._sparsetools.get_csr_submatrix}					
384	0.017	0.000	0.271	0.001	frame.py:11462(_reduce)
1	0.014	0.014	0.014	0.014	{built-in method
scipy.sparse._sparsetools.csr_eliminate_zeros}					
17	0.011	0.001	10.656	0.627	
_rank_genes_groups.py:277(wilcoxon)					
56580/56541	0.011	0.000	0.018	0.000	{built-in method
builtins.isinstance}					
1	0.009	0.009	2.164	2.164	
_rank_genes_groups.py:174(_basic_stats)					
2592	0.006	0.000	0.006	0.000	{built-in method __new__ of
type object at 0x1552446f3fe0}					
384	0.005	0.000	0.074	0.000	nanops.py:1499(_maybe_null_out)
407	0.005	0.000	0.027	0.000	base.py:475(__new__)
520	0.005	0.000	0.005	0.000	{method 'nonzero' of
'numpy.ndarray' objects}					

99	0.004	0.000	0.004	0.000	{built-in method posix.stat}
676	0.004	0.000	0.005	0.000	base.py:5385(__getitem__)
20941/15792	0.004	0.000	0.006	0.000	{built-in method
builtins.len}					
1268	0.004	0.000	0.005	0.000	generic.py:6258(__finalize__)
391	0.004	0.000	0.006	0.000	
utils.py:239(maybe_convert_indices)					
16	0.004	0.000	0.004	0.000	
_continuous_distns.py:370(_norm_cdf)					
500	0.004	0.000	0.018	0.000	
construction.py:517(sanitize_array)					
1	0.004	0.004	10.756	10.756	
_rank_genes_groups.py:398(compute_statistics)					
384	0.003	0.000	0.007	0.000	
cast.py:123(maybe_convert_platform)					
1968	0.003	0.000	0.003	0.000	{built-in method numpy.empty}
23	0.003	0.000	0.003	0.000	{built-in method _io.open_code}
1298	0.003	0.000	0.004	0.000	generic.py:281(__init__)
11613	0.003	0.000	0.007	0.000	generic.py:42(_instancecheck)
384	0.003	0.000	0.008	0.000	range.py:1148(take)
856/855	0.003	0.000	0.799	0.001	indexing.py:1177(__getitem__)
392	0.003	0.000	0.712	0.002	
managers.py:642(reindex_indexer)					
80	0.003	0.000	0.005	0.000	multi.py:1194(_engine)
11613	0.003	0.000	0.004	0.000	generic.py:37(_check)
23	0.003	0.000	0.003	0.000	{method '__exit__' of
'_io._IOBase' objects}					
852	0.003	0.000	0.003	0.000	{method 'copy' of
'numpy.ndarray' objects}					
200	0.003	0.000	0.003	0.000	{built-in method builtins.max}
401	0.003	0.000	0.003	0.000	cast.py:550(maybe_promote)
853	0.002	0.000	0.774	0.001	indexing.py:1720(_getitem_axis)
1170	0.002	0.000	0.003	0.000	base.py:650(_simple_new)
23	0.002	0.000	0.002	0.000	{method 'read' of
'_io.BufferedReader' objects}					
78	0.002	0.000	0.002	0.000	{built-in method numpy.zeros}
429	0.002	0.000	0.003	0.000	
cast.py:1166(maybe_infer_to_datetimelike)					
5	0.002	0.000	0.008	0.002	records.py:588(fromarrays)
384	0.002	0.000	0.274	0.001	
generic.py:12524(_min_count_stat_function)					